Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 7
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



In cursul anterior

- Comutatoare Ierarhice
- Comutatoare Neierarhice
- Exemple de Comutatoare Procesor Memorie
- Interconectarea in Sistemele cu Resurse Multiple
- Retele de Comutare Ierarhice
- Retele de Comutare de tip Delta
- Retele Bazate pe Rutare
- Performantele Retelelor de Comutare



Cuprins

- The Roofline Model
- Architecture
- Programming
- Performance
- Going Forward

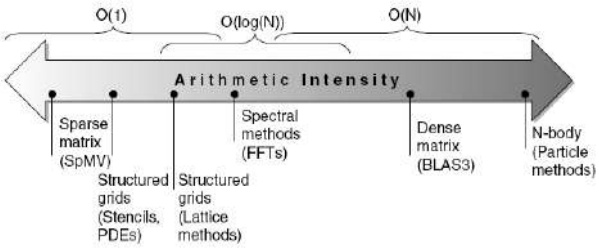




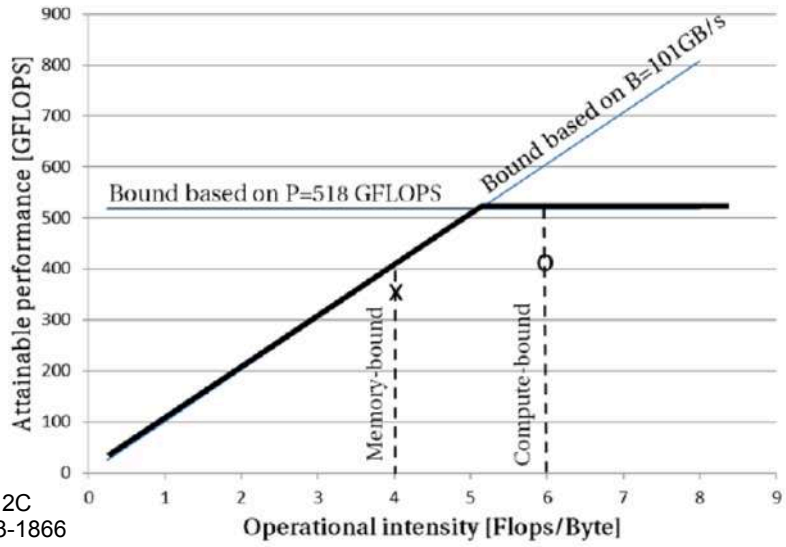
The Roofline Model

The Roofline Performance Model

- Basic Idea
  - Plot peak floating-point throughput as a function of arithmetic intensity
  - Ties together floating-point performance and memory performance for a target machine
- Arithmetic Intensity
  - Floating-point operations per byte read



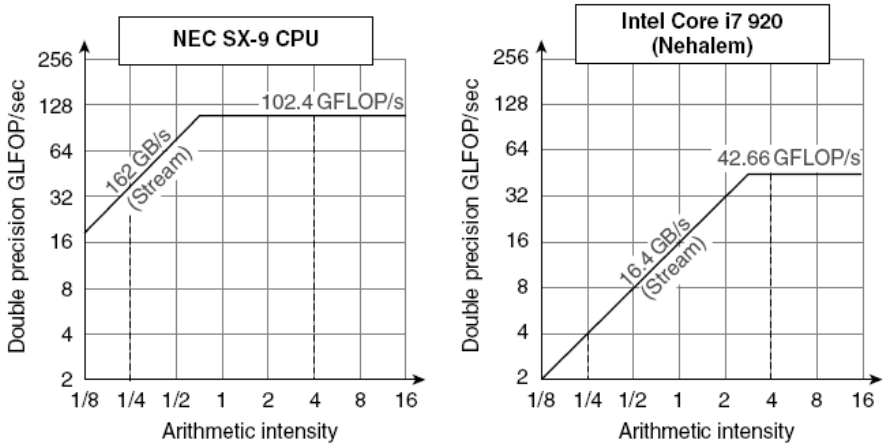
Modelul Roofline



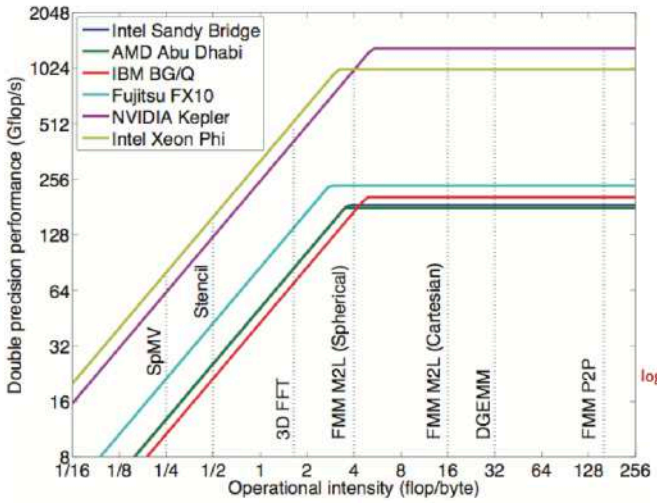
Intel XeonE5-2697v2 12C
2.7GHz cu memorie DDR3-1866

Roofline Performance Examples

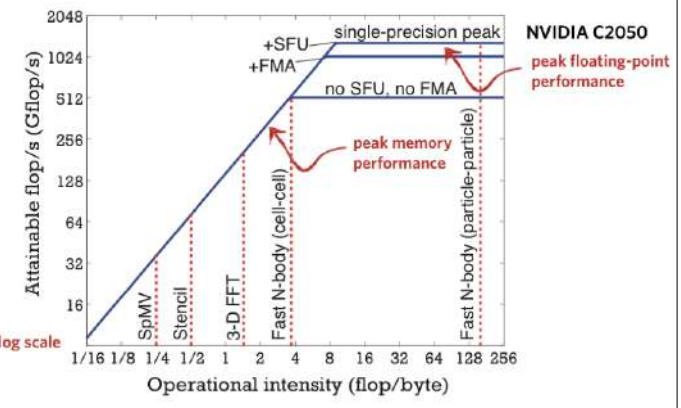
- Attainable GFLOPs/sec Min = (Peak Memory BW × Arithmetic Intensity, Peak Floating Point Performance)



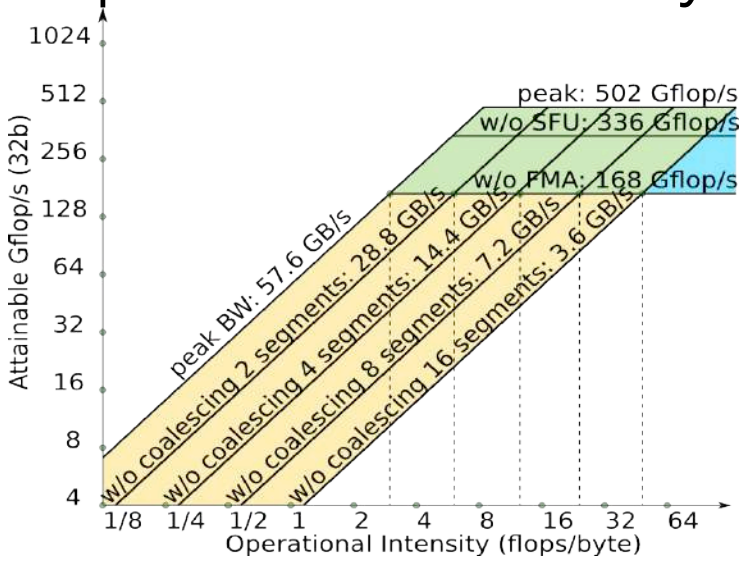
Roofline Performance Examples



log/log scale

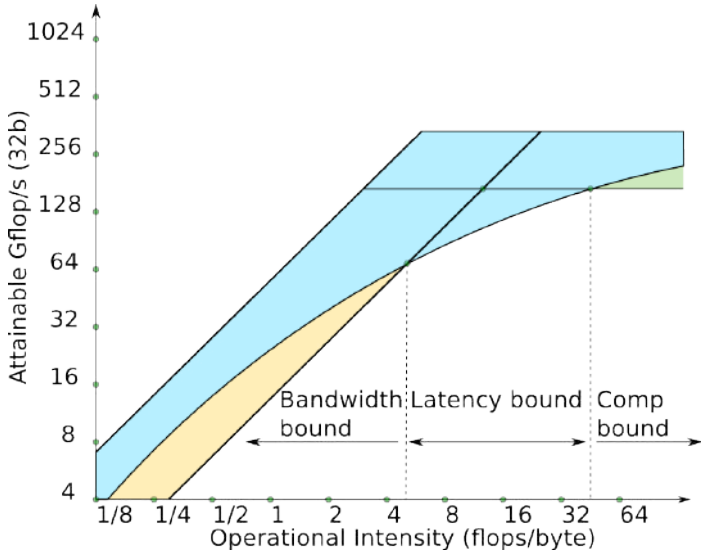


Optimization is Key



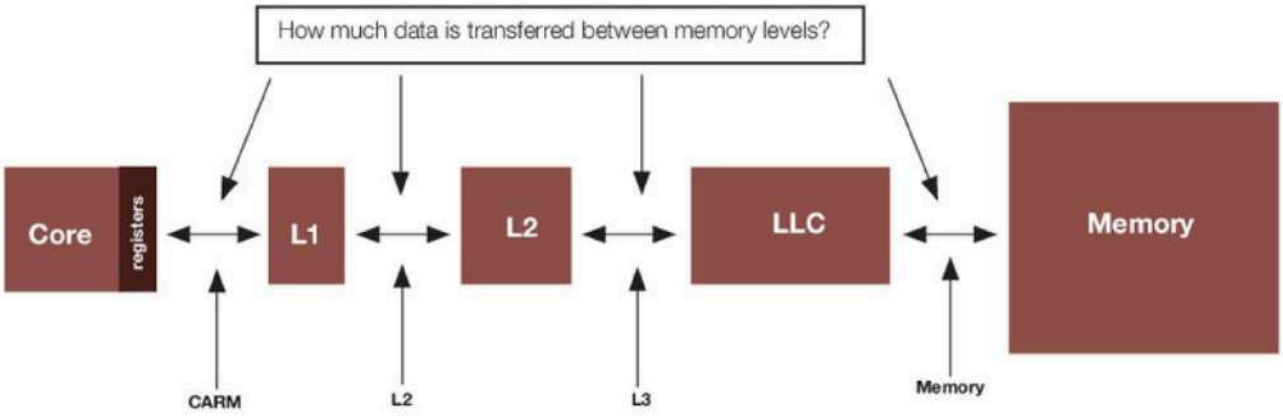


Bottlenecks

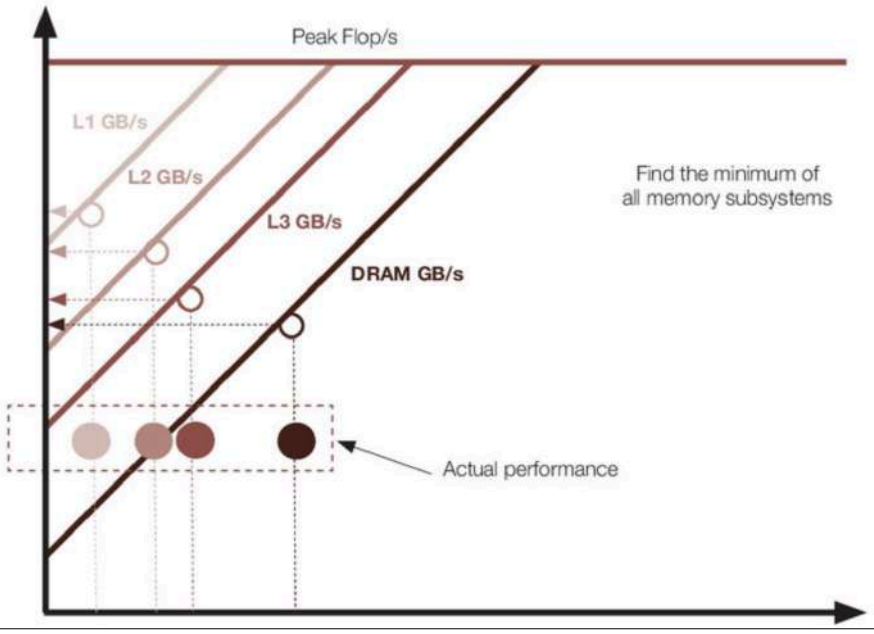




Cache-Aware Roofline Model



Performance Limitations

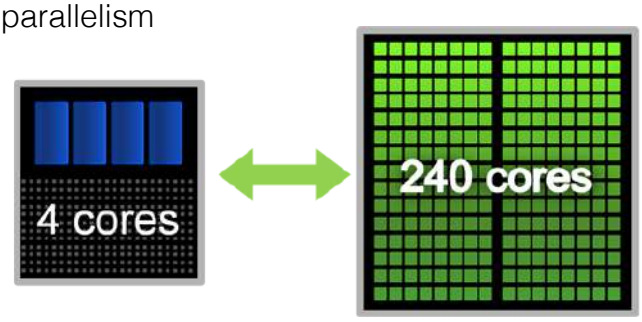




Architecture

CPUs & GPUs

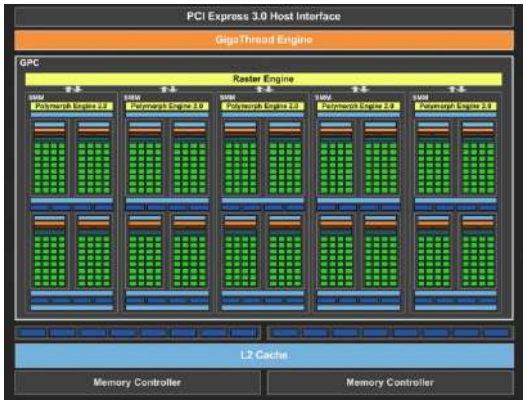
- Typically GPUs and CPUs coexist in a heterogeneous setting
- “Less” computationally intensive part runs on CPU
  - Coarse-grained parallelism
- More intensive parts run on GPU
  - Fine-grained parallelism



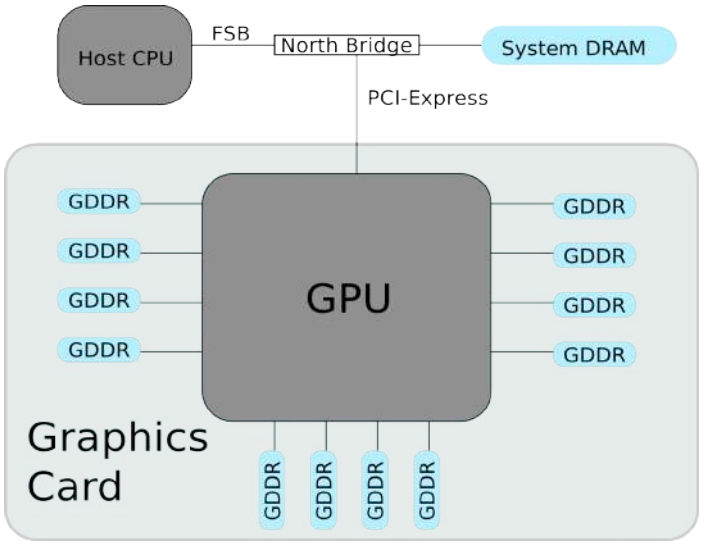
NVIDIA Kepler K40

- 2880 Streaming Processors/cores (SPs) organized as 15 Streaming Multiprocessors (SMs)
- Each SM contains 192 cores
- Memory size of the GPU system: 12 GB
- Clock speed of a core: 745 MHz

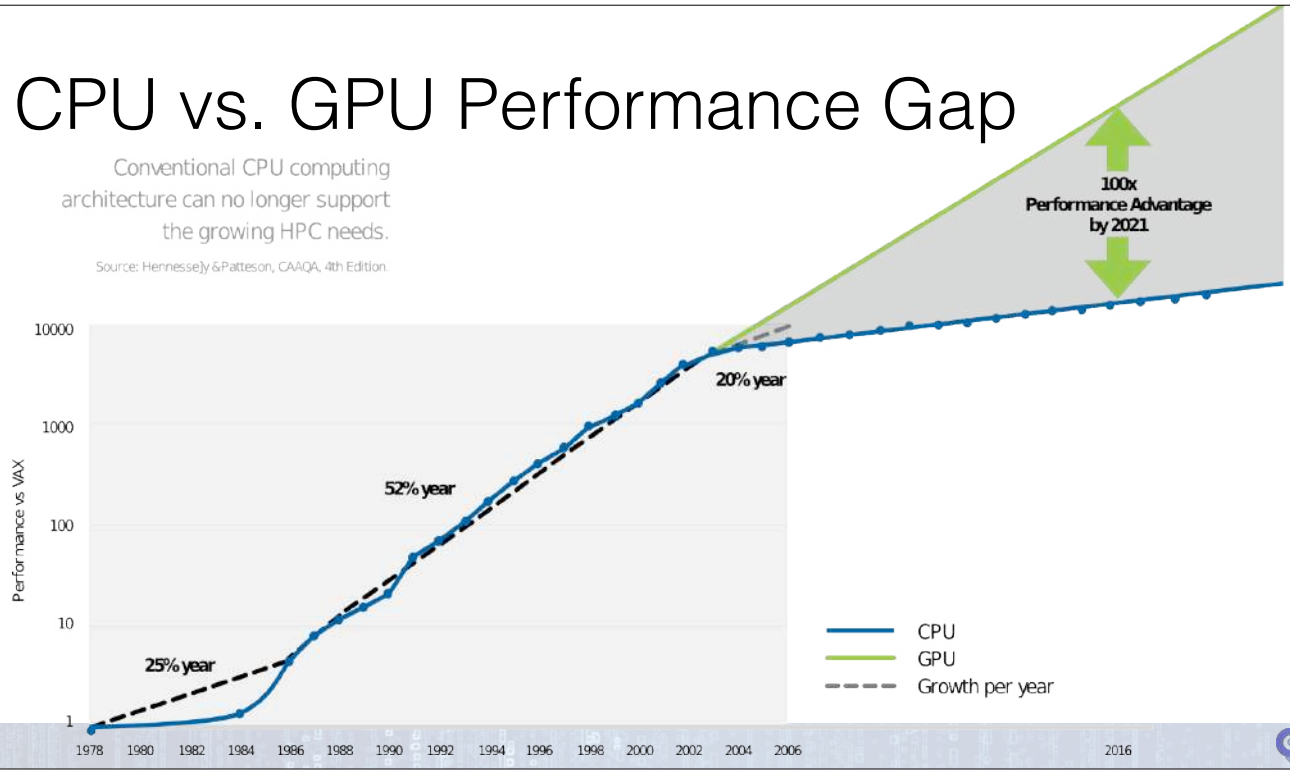




System Architecture



CPU vs. GPU Performance Gap

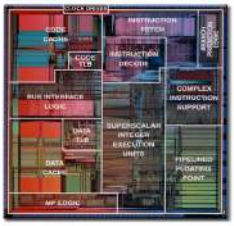




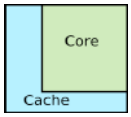
CPU/APU Architectures

Pentium Processors Evolution

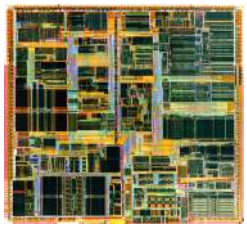
Pentium I

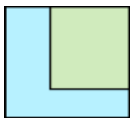


Chip area breakdown

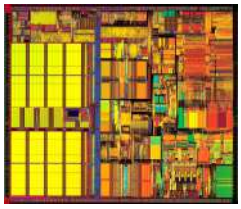


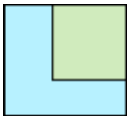
Pentium II



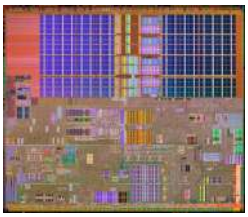


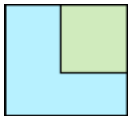
Pentium III





Pentium IV



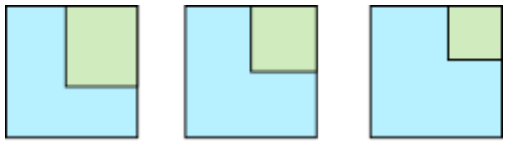


Q: What can you observe? Why?



Extrapolation of Single Core CPUs

If we extrapolate the trend, in a few generations, Pentium will look like this:





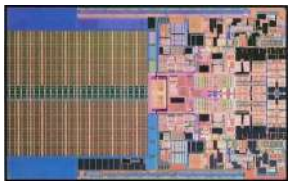
Of course, we know it did not happen.

Q: What happened instead? Why?

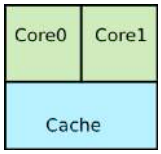


Multi-Core CPUs Evolution

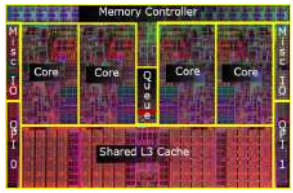
Penny

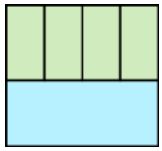


Chip area breakdown

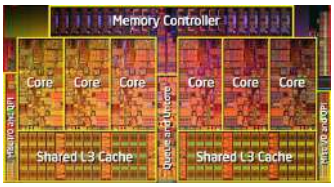


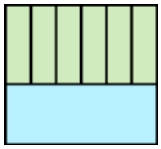
Bloomfield



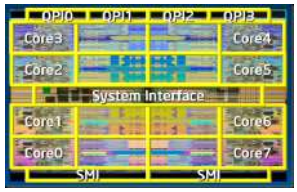


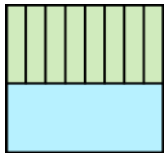
Gulftown





Beckton

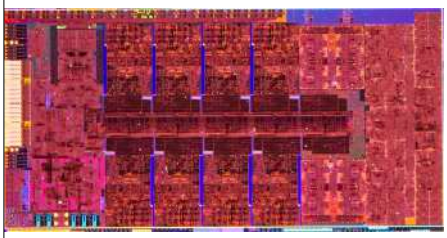




Q: What can you observe? Why?



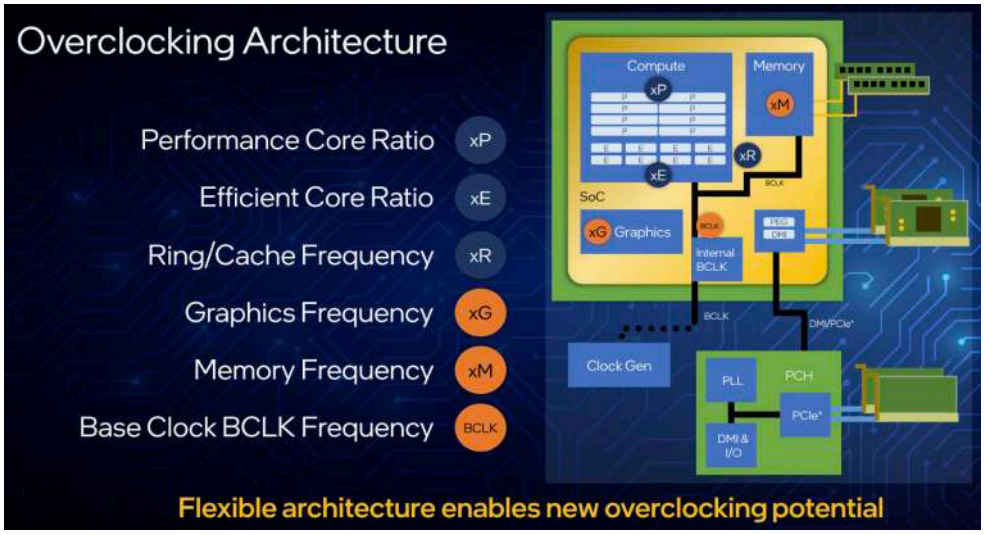
Intel Core i9-12900K



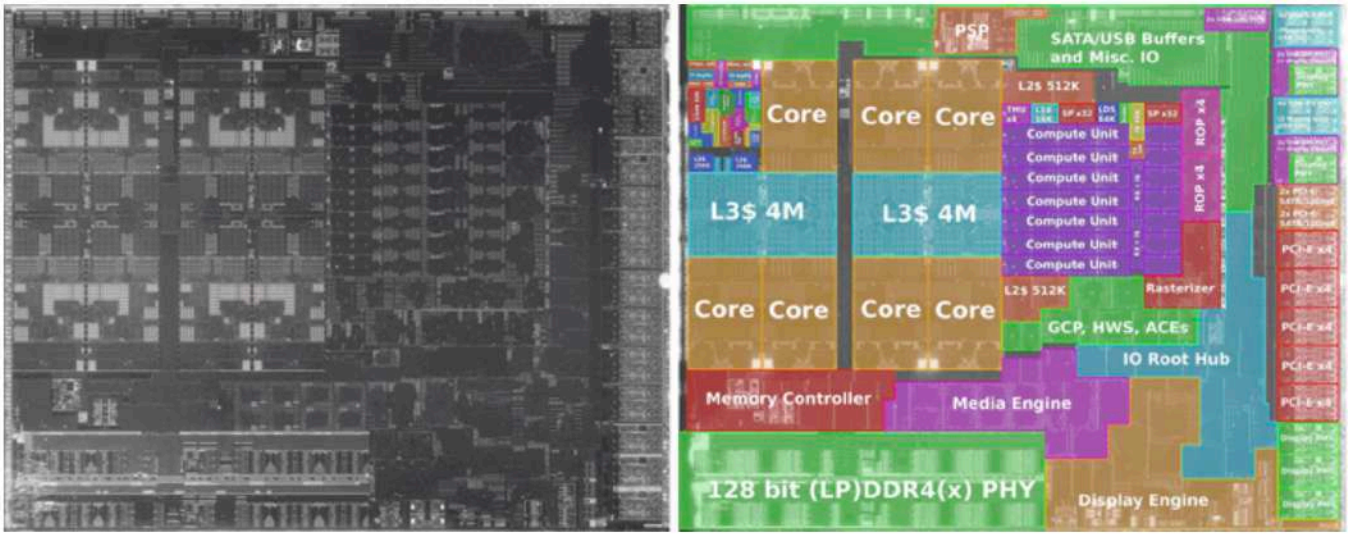




Intel Overclocking Architecture



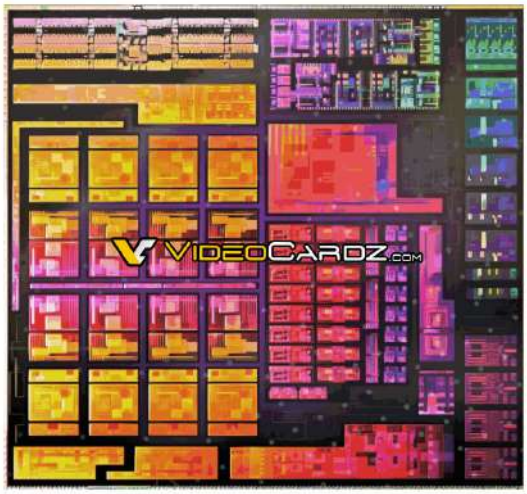
AMD Ryzen Architecture



AMD Ryzen Zen2/Zen3 Architectures

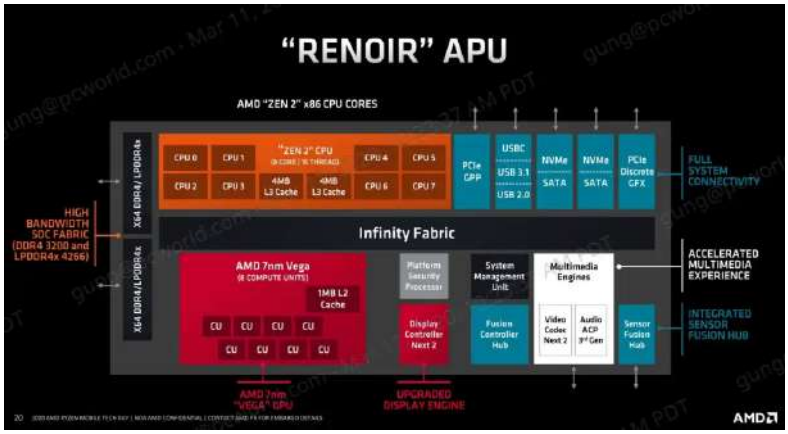


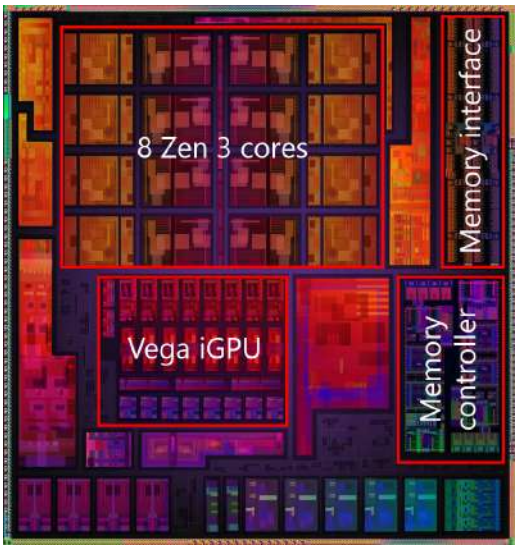
AMD Renoir (Zen2)



AMD Cezanne (Zen3)

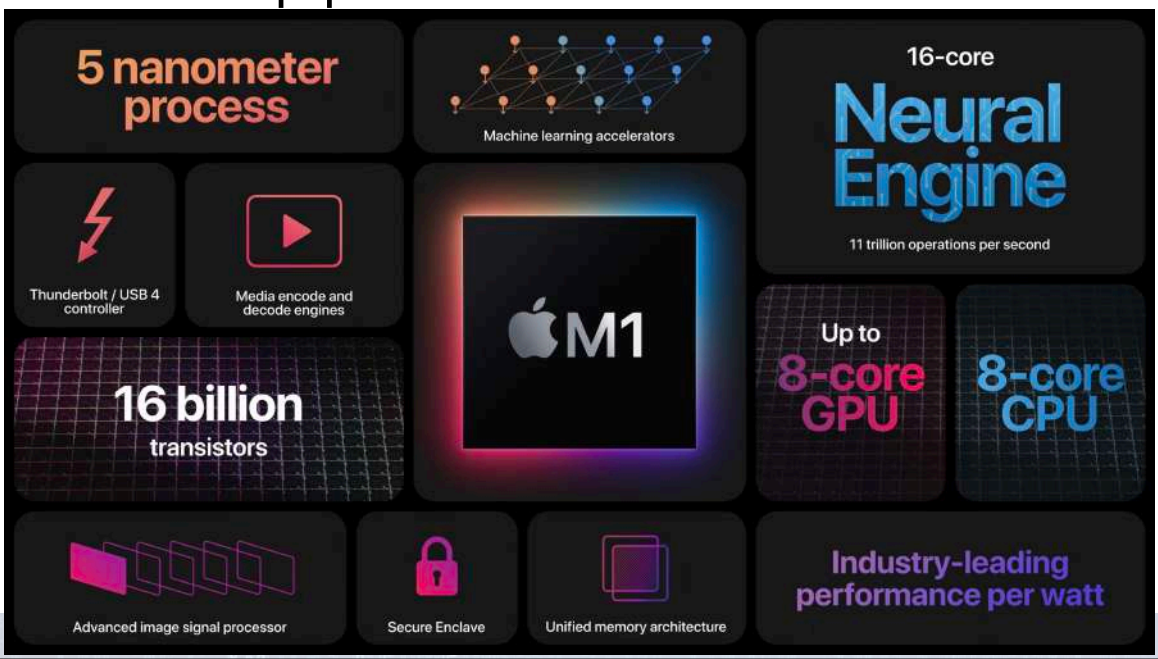
AMD Ryzen Zen2/Zen3 Architectures



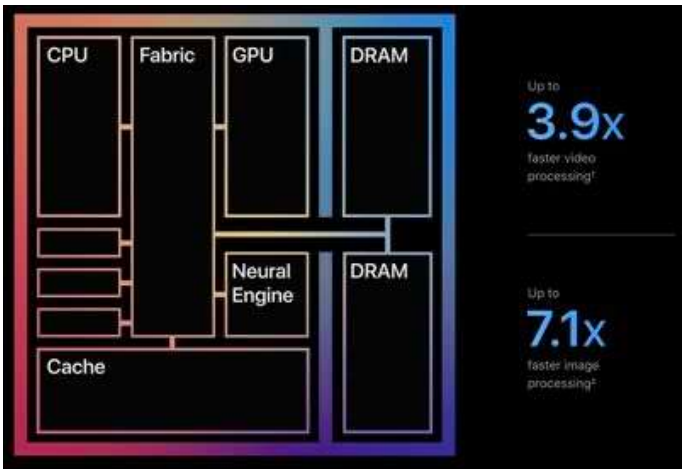


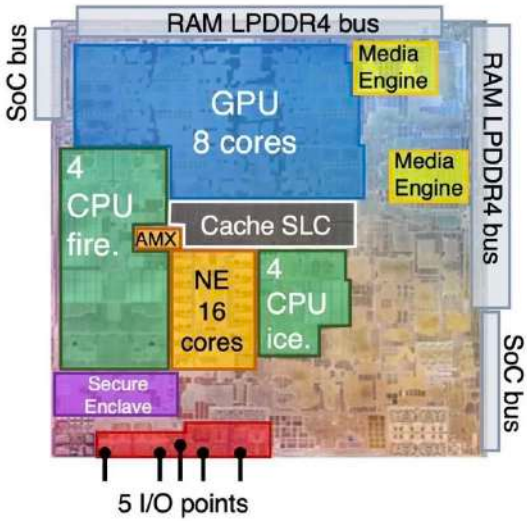


Apple M1 Processor



Apple M1 Architecture



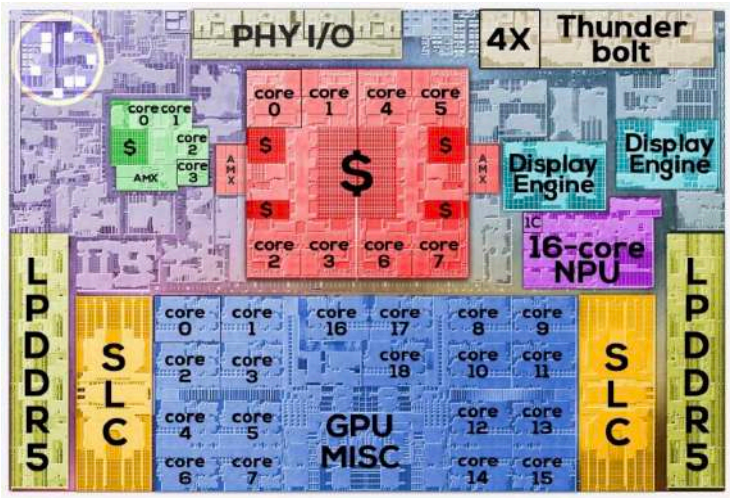


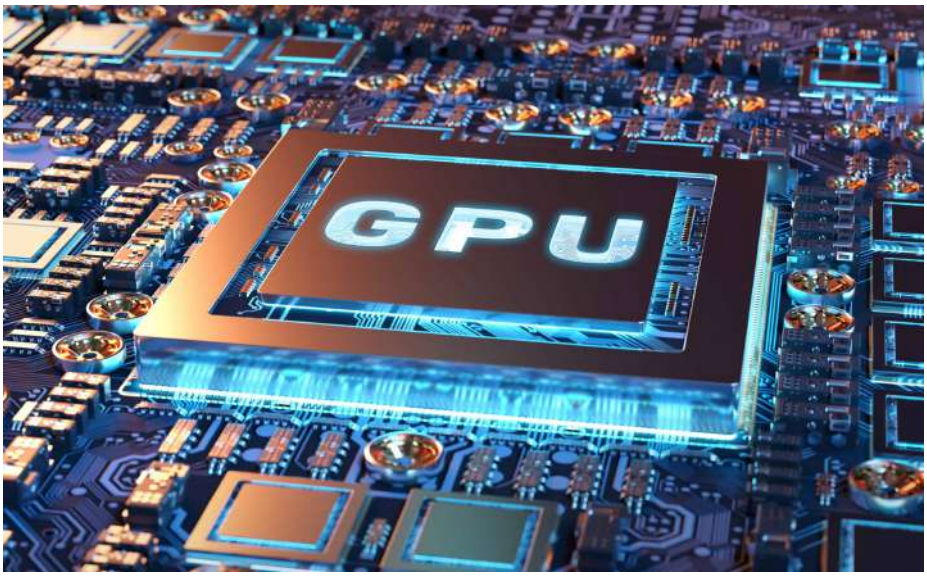
Apple M2 Processor



Apple M2 Architecture

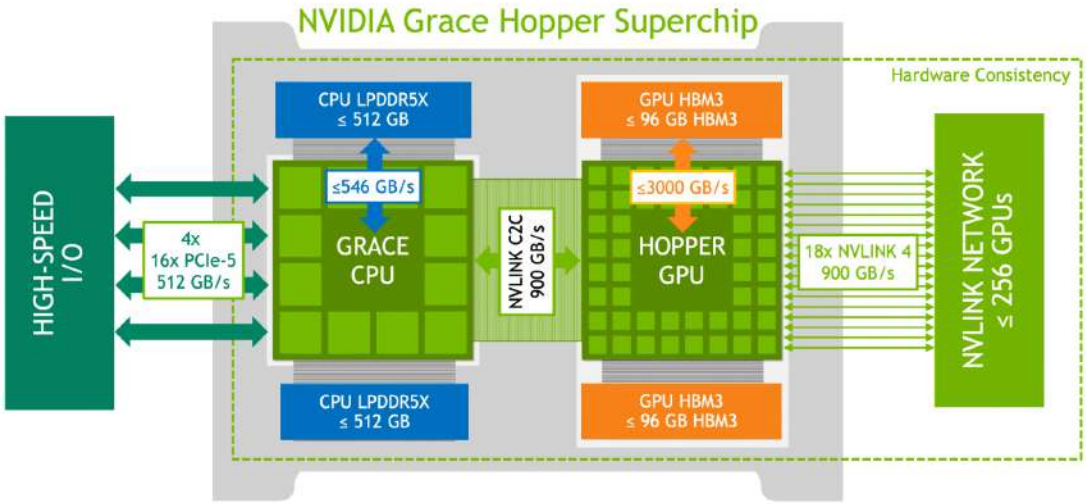






GPU Architectures

NVidia Grace/Hoper Superchip

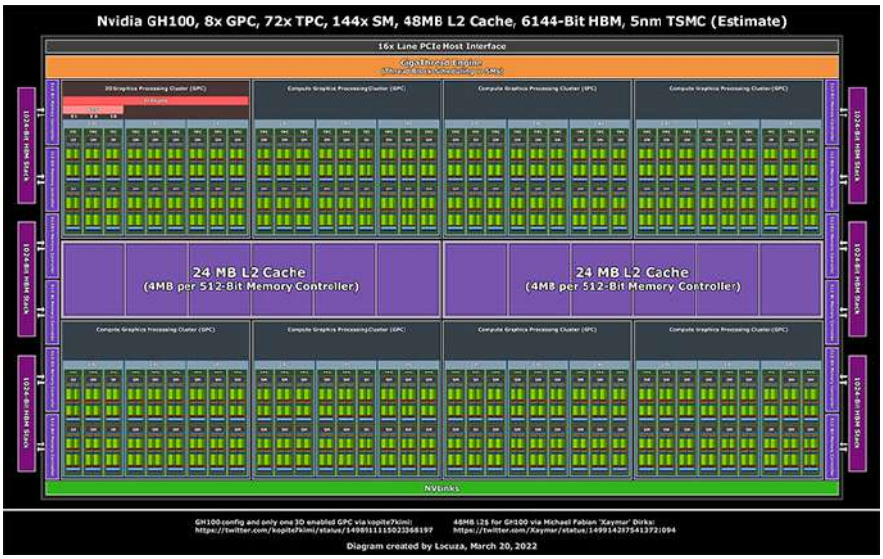


NVidia 144-core Grace CPU (Arm v9)



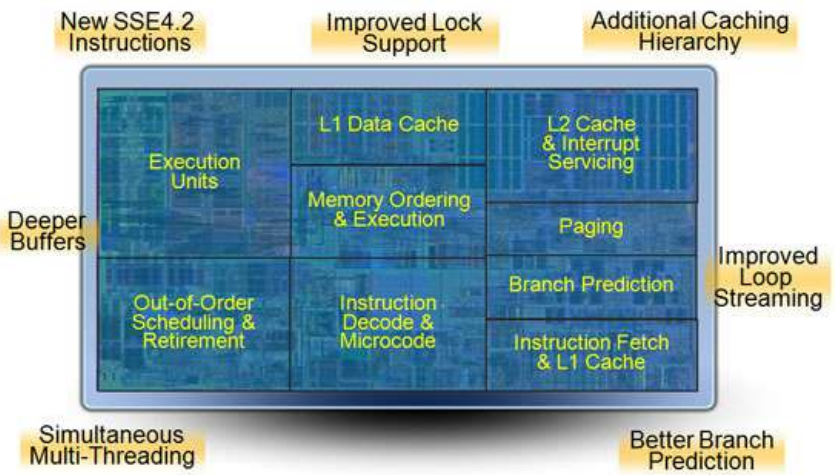


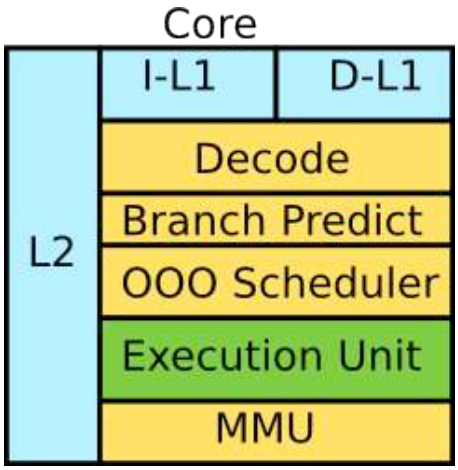
NVidia Hopper GPU





Let's Zoom(in)to an IA Core



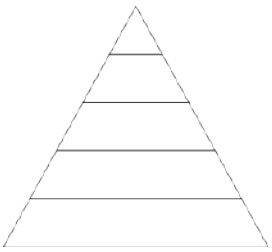


Less than **10%** of total chip area is used for the real execution.

Q: Why?



Revisiting the Memory Hierarchy



	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	1K	10	1	1000
L1	32K	20	5	100
L2	256K	100	10	100
L3	8M	200	50	100
Off-chip	4G	2000	100	10

Notes on Energy at 45nm:
64-bit Int ADD takes about 1 pJ.
64-bit FP FMA takes about 200 pJ.

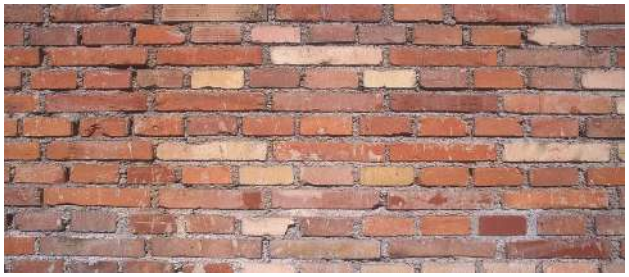
It seems we can not further increase the computational density!



The Brick Wall

Power Wall: power expensive, transistors “free”
Memory Wall: Memory slow, multiplies fast
ILP Wall: diminishing returns on more ILP HW

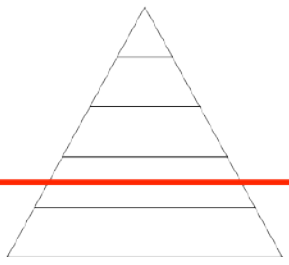
Power Wall + Memory Wall + ILP Wall = Brick Wall



Q: How to Break the Wall?
Exploit the parallelism inside the application…



Step 1: Trade Latency with Throughput



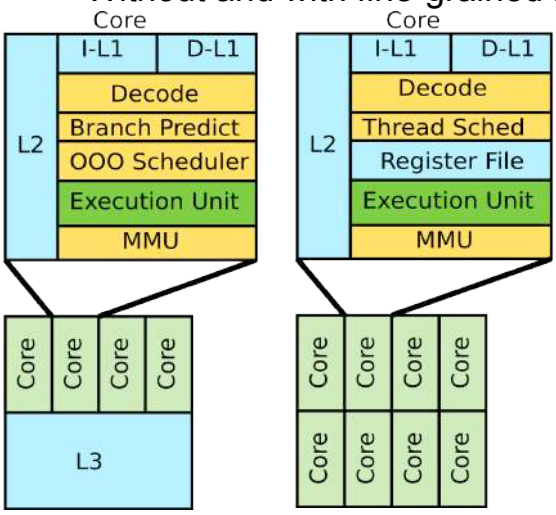
	Size (Byte)	Energy (pJ)	Delay (cycles)	Bandwidth (GB/s)
Reg	~~1K~~ 16K	~~10~~ 20	~~1~~ 2~3	1000
L1	32K	20	5	100
L2	256K	100	10	100
~~L3~~	~~8M~~	~~200~~	~~50~~	~~100~~
Off-chip	4G	2000	100	10

Hide the memory latency through fine-grained interleaved threading



Fine-Grained Interleaved Threading

Without and with fine-grained interleaved threading



Pros:
Reduce cache size
No branch predictor
No OOO scheduler

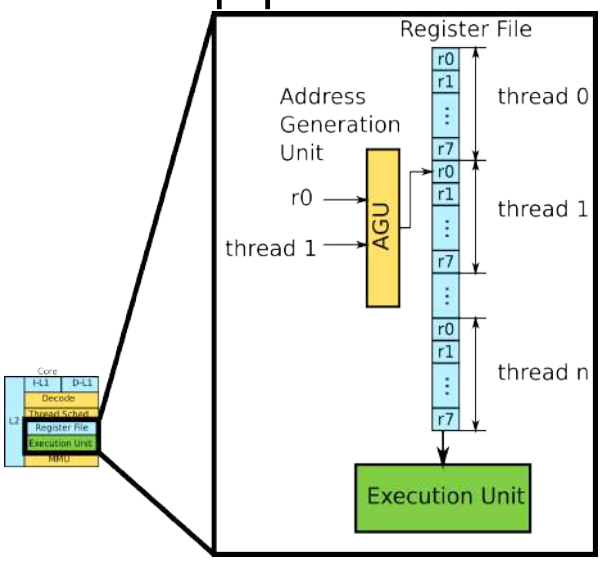
Cons:
Register pressure
Thread scheduler
Requires huge parallelism

OOO = Out-of-order Execution



Hardware Support

Register file supports zero overhead context switch between interleaved threads

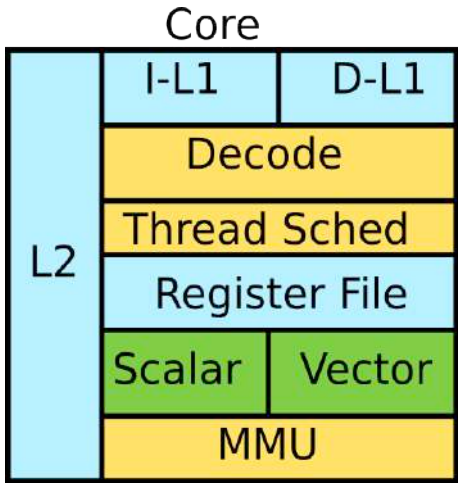




Can we make Further Improvements?

- Reducing large cache gives 2x computational density.
- Q: Can we make further improvements?

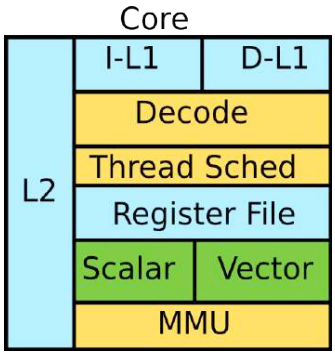
Hint:
We have only utilized thread level parallelism (TLP) so far.



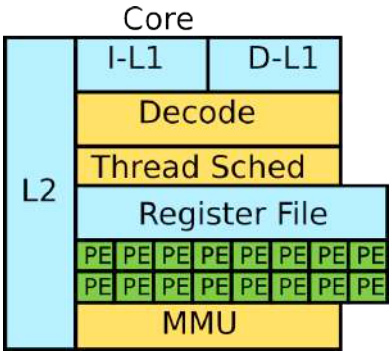
Step 2: Single Instruction Multiple Data

GPU uses wide SIMD: 8/16/24/... processing elements (PEs)
CPU uses short SIMD: usually has vector width of just 4

SSE has 4 data lanes

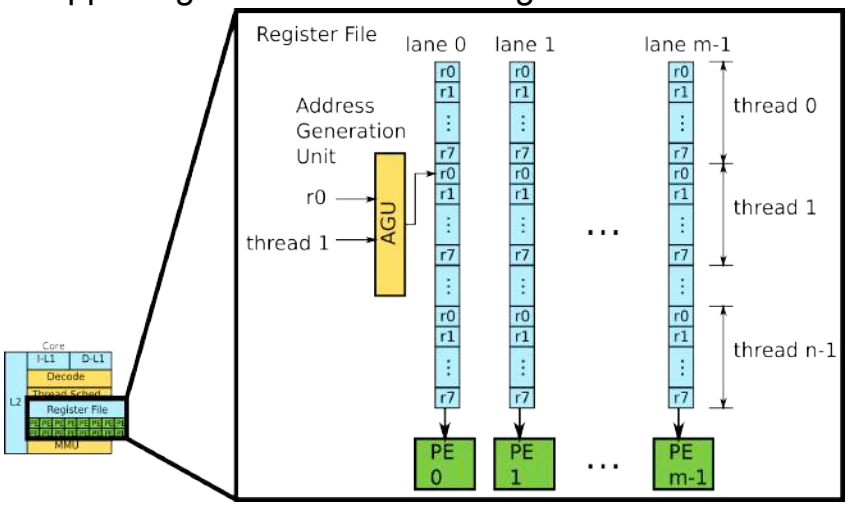


GPUs have 8/16/24/... data lanes



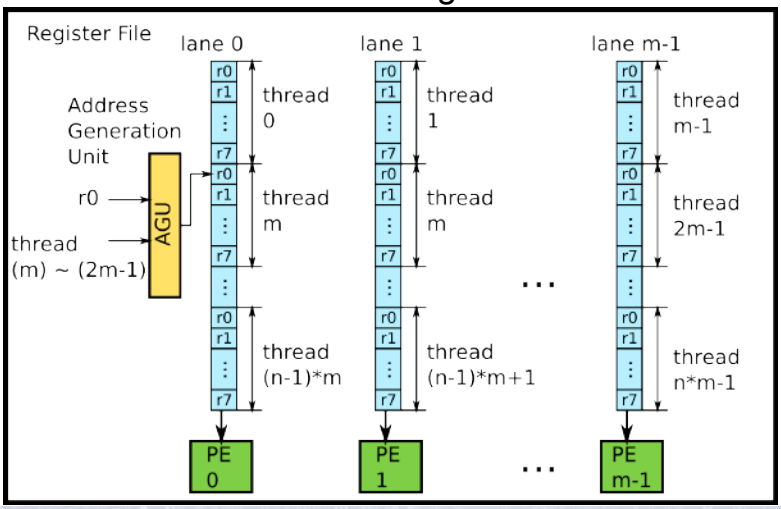
Hardware Support

Supporting interleaved threading + SIMD execution



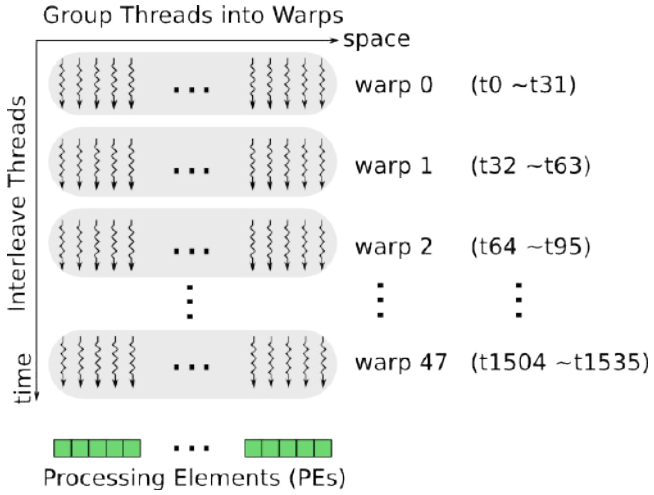
Single Instruction Multiple Thread (SIMT)

Hide vector width using scalar threads



Example of SIMT Execution

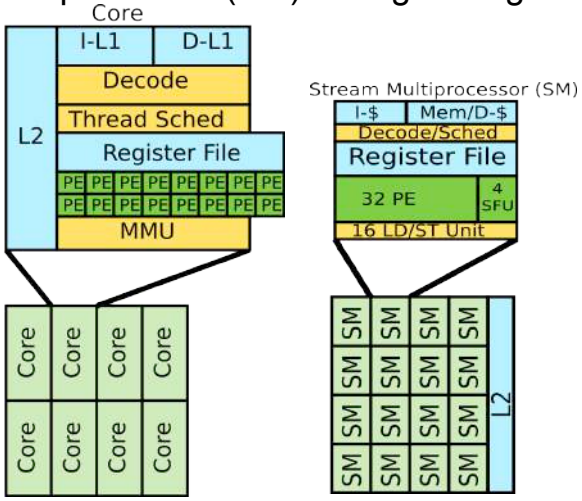
Assume 32 threads are grouped into one warp.





Step 3: Simple Core

Stream Multiprocessor (SM) is a light-weight core compared to a “classical” IA core



Light weight PE:

Fused Multiply Add (FMA)

SFU:

Special Function Unit



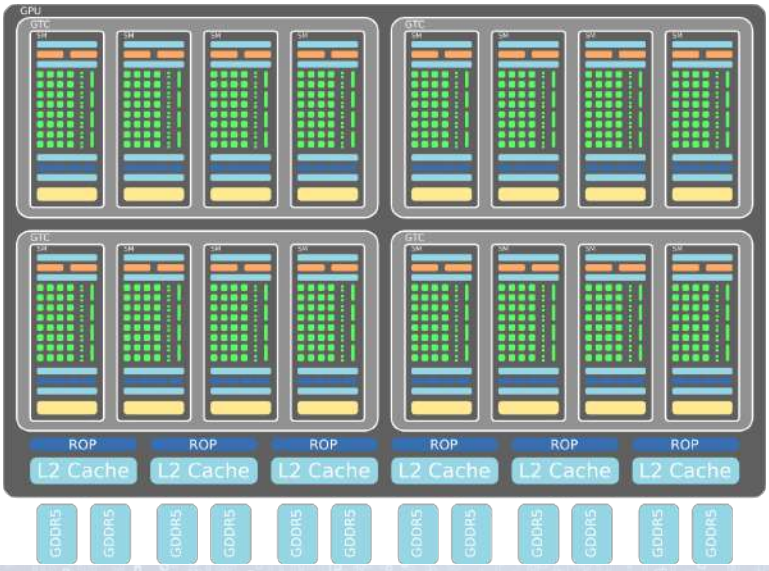
NVIDIA's Motivation of a Simple Core



“The [multiple IA-core] approach is analogous to trying to build an airplane by putting wings on a train."

Bill Dally, NVIDIA

How Do We Reach Here?

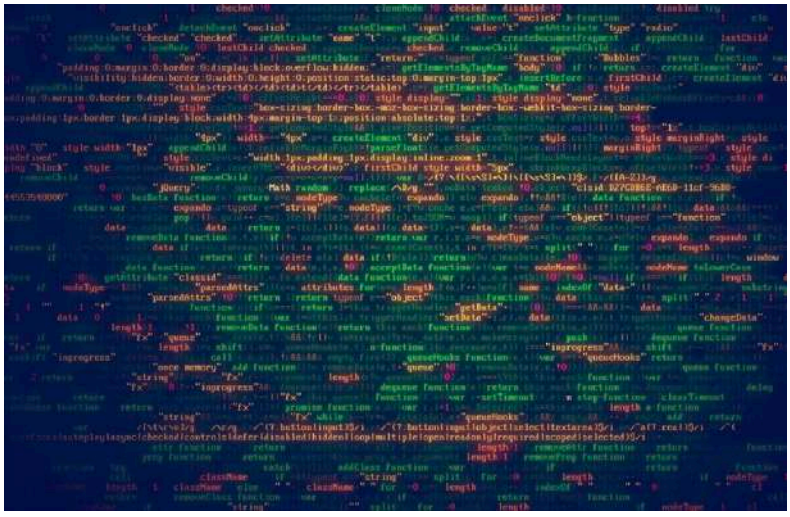


NVIDIA Fermi, 512 Processing Elements (PEs)

Throughput Oriented Architectures

- Fine-grained interleaved threading (~2x comp density)
- SIMD/SIMT (>10x comp density)
- Simple core (~2x comp density)

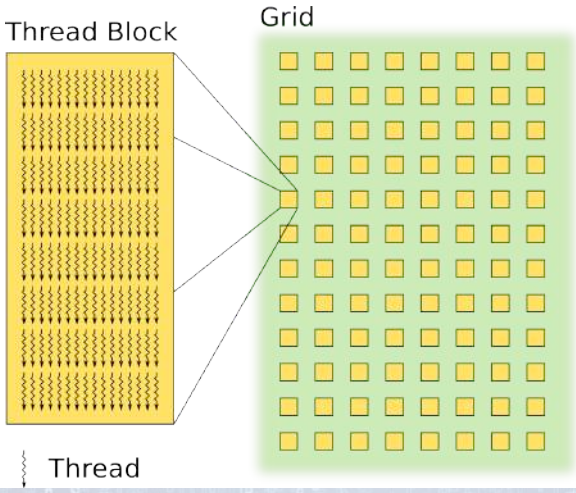




Programming

Multi-Threading at another Level

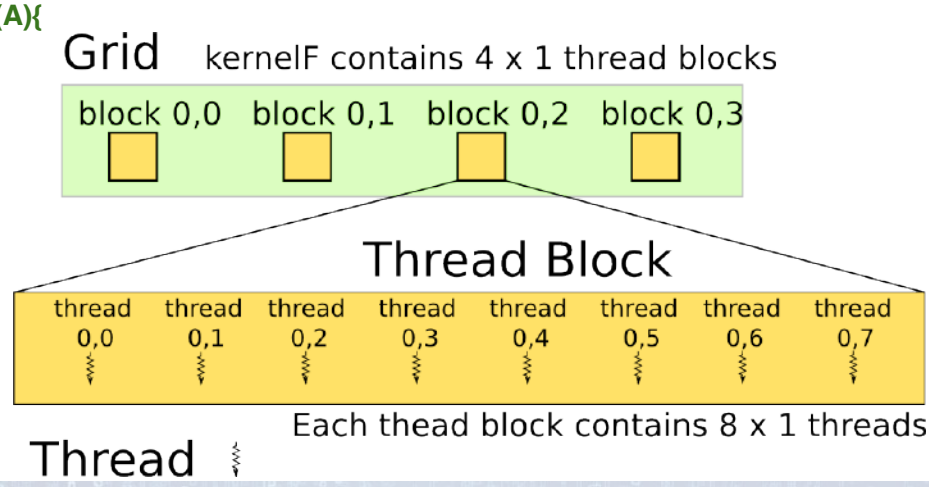
- Massive number (>10000) of **light-weight** threads



Two Levels of Thread Hierarchy

```
kernelF<<<(4,1),(8,1)>>>(A);

__device__   kernelF(A){
   i = blockIdx.x;
   j = threadIdx.x;
   A[i][j]++;
}
```

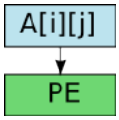


Vector Programming

- Vector width is exposed to programmers

Scalar program

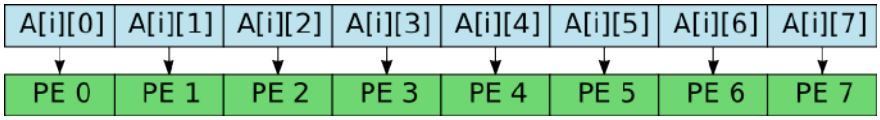
```
float A[4][8];
do-all(i=0;i<4;i++){
   do-all(j=0;j<8;j++){
      A[i][j]++;
   }
}
```



Vector program (vector width of 8)

```
float A[4][8];

do-all(i=0;i<4;i++){
   movups xmm0, [ &A[i][0] ]
   incps xmm0
   movups [ &A[i][0] ], xmm0
}
```





GPU Program

- Program expresses data level parallelism (DLP) in terms of thread level parallelism (TLP)
- Hardware converts TLP into DLP at run time

Scalar program

```
float A[4][8];
do-all(i=0;i<4;i++){
   do-all(j=0;j<8;j++){
      A[i][j]++;
   }
}
```

GPU program

```
float A[4][8];

kernelF<<<(4,1),(8,1)>>>(A);

__device__    kernelF(A){
   i = blockIdx.x;
   j = threadIdx.x;
   A[i][j]++;
}
```

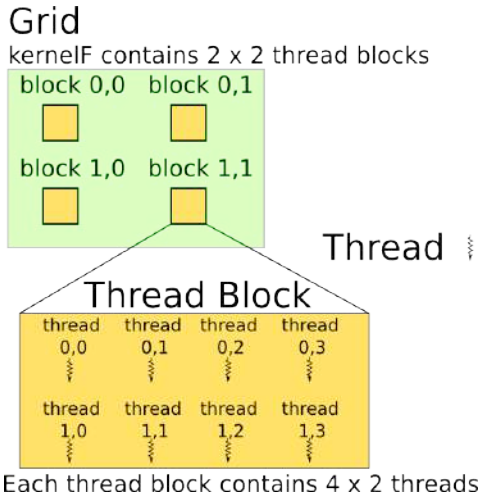



Multi-dimension Thread and Block ID

- Both grid and thread block can have two dimensional index

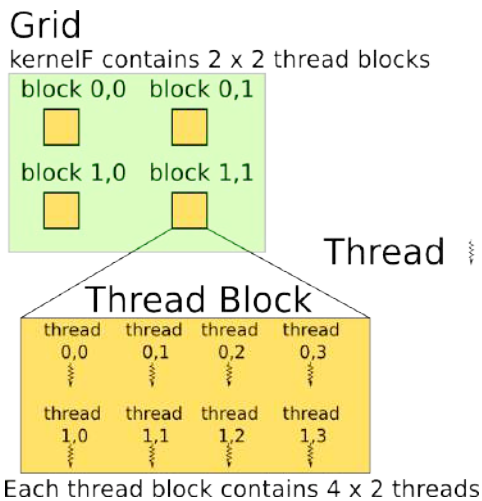
```
kernelF<<<(2,2),(4,2)>>>(A);

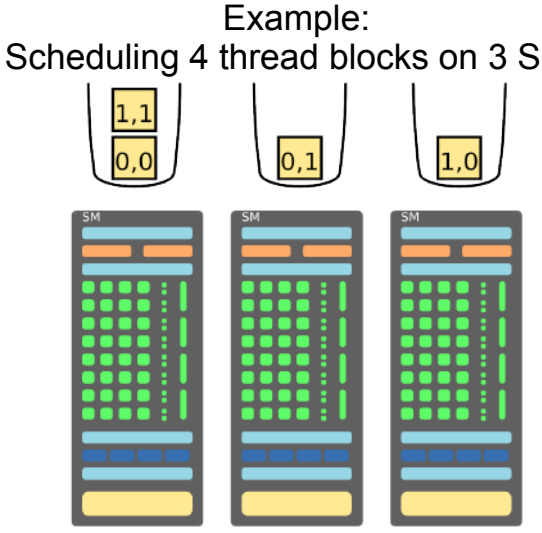
__device__    kernelF(A){
   i = blockDim.x * blockIdx.y
      + blockIdx.x;
   j = threadDim.x * threadIdx.y
      + threadIdx.x;
   A[i][j]++;
}
```





Scheduling Thread Blocks on SM





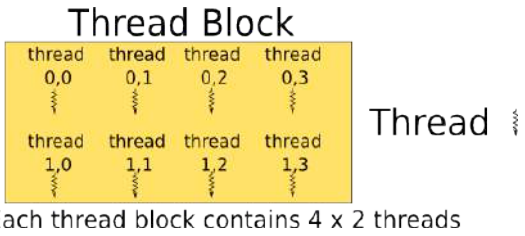


Executing Thread Block on SM

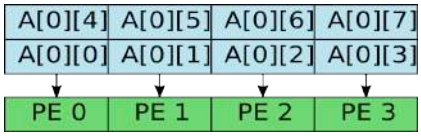
```
kernelF<<<(2,2),(4,2)>>>(A);

__device__    kernelF(A){
   i = blockDim.x * blockIdx.y
      + blockIdx.x;
   j = threadDim.x * threadIdx.y
      + threadIdx.x;
   A[i][j]++;
}
```

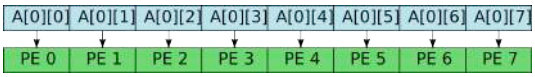
Note: The number of Processing Elements (PEs) is transparent to programmer



Executed on machine with width of 4:

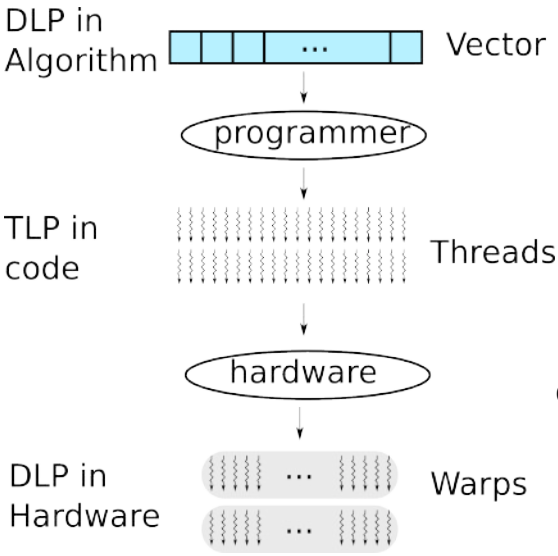


Executed on machine with width of 8:





Programmers Think in Threads



Q: Why make this hassle?



Why Use Thread instead of Vector?

- Thread **Pros**:
  - Portability: Machine width is transparent in ISA (Instruction Set Architecture)
  - Productivity: Programmers don't take care of vector width on processor arch
- Thread **Cons**:
  - Manual sync: Give up lock-step within vector
  - Scheduling of threads could be inefficient
  - Debug: "Threads considered harmful". Thread programming is notoriously hard to debug
- Programmers **explicitly express DLP in terms of TLP**
- Programmers **explicitly manage memory hierarchy**



Hierarchical Parallelism

- Parallel computations arranged as grids
- One grid executes after another
- Grid consists of blocks
- Blocks assigned to SM
  - A single block assigned to a single SM
  - Multiple blocks can be assigned to a SM
- Block consists of elements
- Elements computed by threads



Thread Blocks

- Thread block
  - Array of concurrent threads that execute the same program and can cooperate to compute the result
  - Consists of up to 1024 threads (K40) - hp-sl.q queues / 2048+ others
  - Has shape and dimensions (1d, 2d or 3d) for threads
- A thread ID has corresponding 1d, 2d or 3d indices
- Each SM executes up to 16 thread blocks concurrently (K40)
- Threads of a thread block share memory
- Maximum threads per SM: 2048 (K40)



General GPU Programming Steps

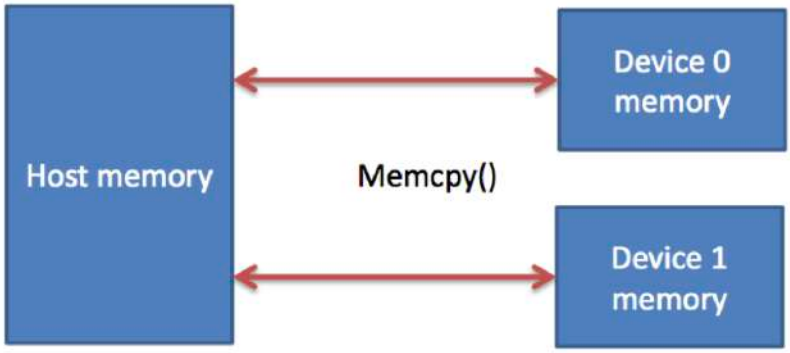
1. Initialize the data on the CPU / space on the GPU
2. Copy data from CPU to GPU
3. Compute on GPU
4. Copy data back from GPU to CPU

- By default, execution on host doesn't wait for kernel to finish
- General rules:
  - **Minimize** data transfer between CPU & GPU
  - **Maximize** number of threads on GPU

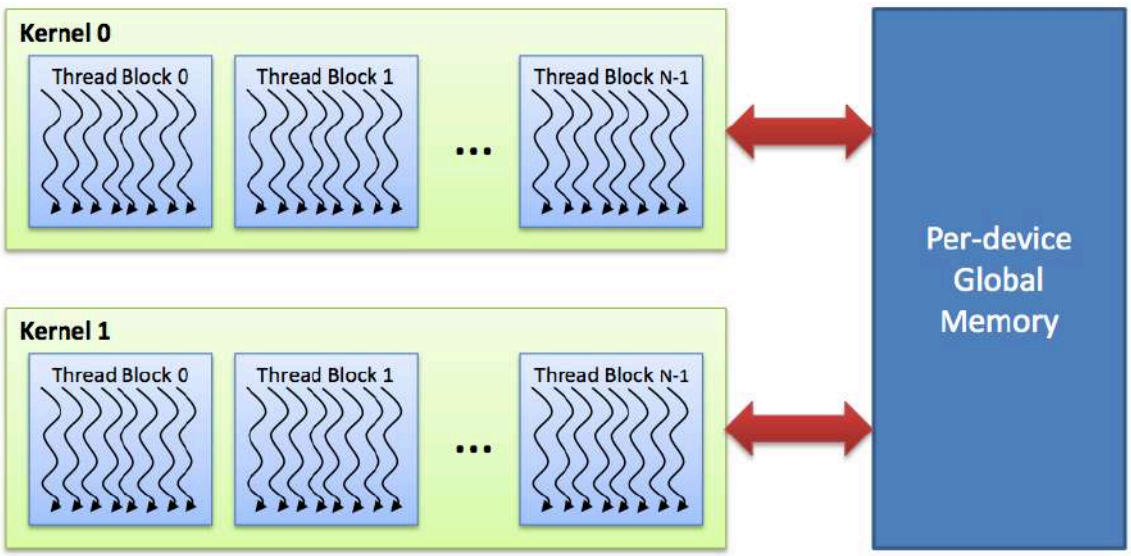


GPU Memory Hierarchy

- Global (device) memory
  - Accessible by all threads as well as host (CPU) – Data lifetime is from allocation to deallocation

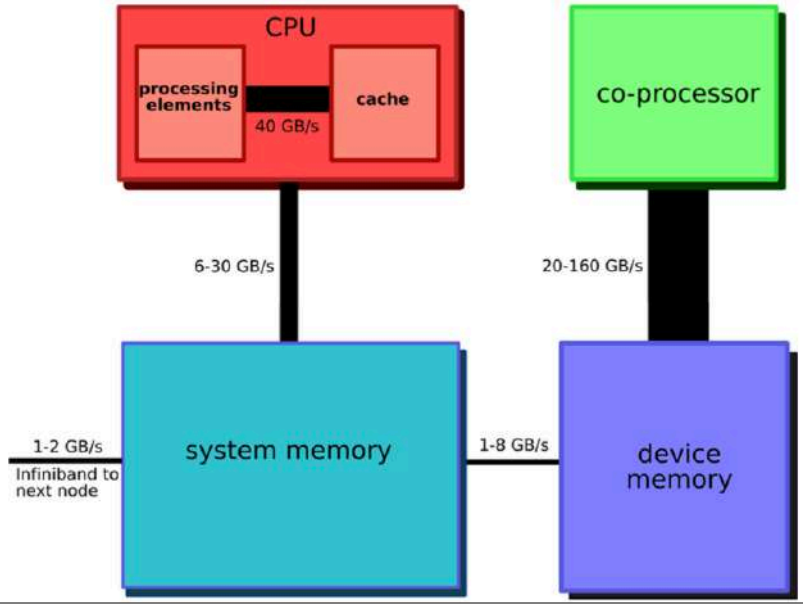


GPU Memory Hierarchy





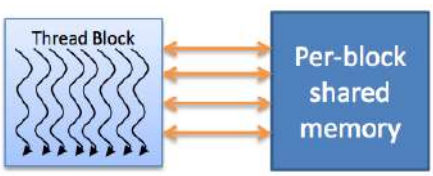
Bandwidth in a CPU-GPU System



GPU Memory Hierarchy

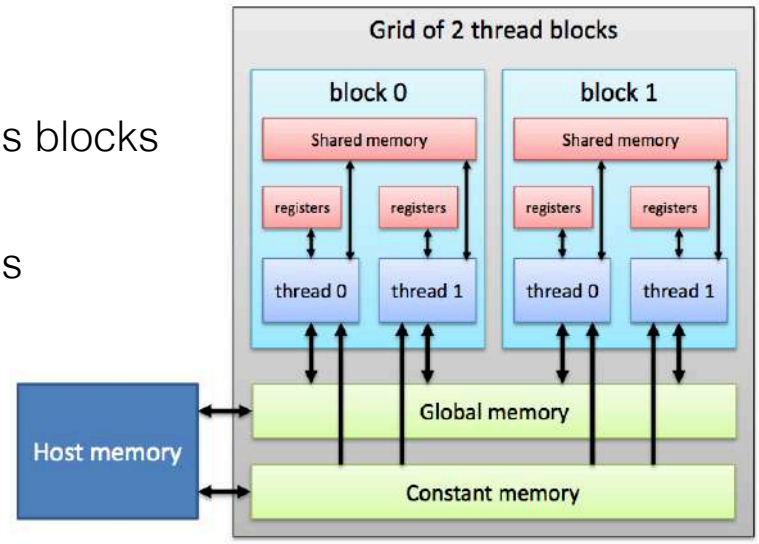
- Local storage
  - Each thread has own its local storage
  - Mostly registers - managed by the compiler
  - Data lifetime = thread lifetime
- Shared memory
  - Each thread block has its own shared memory
    - Accessible only by threads within that block
  - Data lifetime = block lifetime





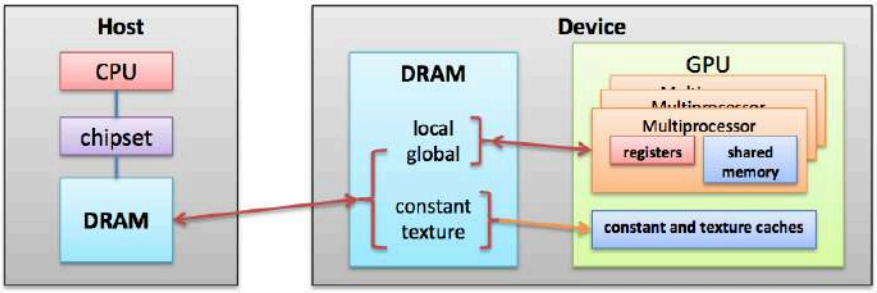
GPU Memory Hierarchy

- 1D grid
  - 2 threads blocks
- 1D block
  - 2 threads





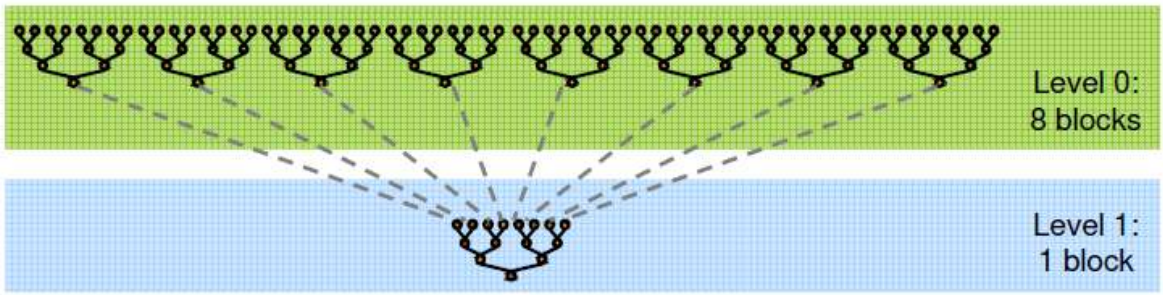
GPU Memory Hierarchy



Memory	Location	Cached	Access	Scope	Lifetime
Register	On-chip	N/A	R/W	One thread	Thread
Local	Off-chip	No	R/W	One thread	Thread
Shared	On-chip	N/A	R/W	All threads in a block	Block
Global	Off-chip	No	R/W	All threads + host	Application
Constant	Off-chip	Yes	R	All threads + host	Application
Texture	Off-chip	Yes	R	All threads + host	Application

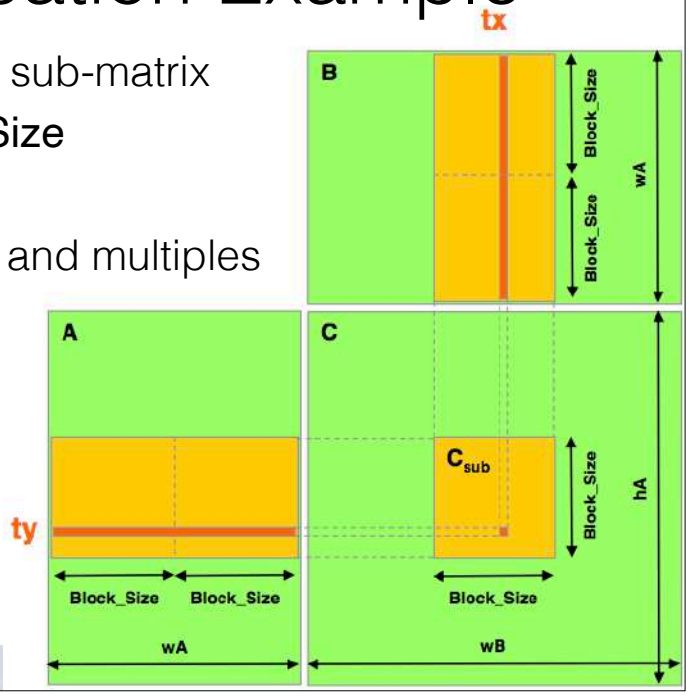


Reduction Example

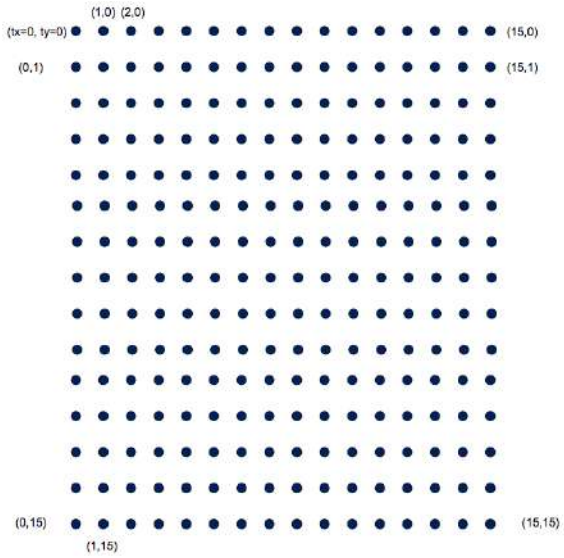


Matrix Multiplication Example

- One block computes one square sub-matrix C_{sub} of size **Block_Size x Block_Size**
- One thread one entry of C_{sub}
- Assume that of A & B are square and multiples of **Block_Size**
  - We will have **Block_Size=16x16**



A Block of 16 X 16 Threads





Matrix Multiplication - Host Code

```
// Thread block size
#define BLOCK_SIZE 16

// Forward declaration of the device multiplication func.
__global__ void Muld(float*, float*, int, int, float*);

// Host multiplication function
// Compute C = A * B
// hA is the height of A
// wA is the width of A
// wB is the width of B
void Mul(const float* A, const float* B, int hA, int wA, int wB, float* C) {
 int size;

  // Load A and B to the device
 float* Ad;
 size = hA * wA * sizeof(float);
 cudaMalloc((void**)&Ad, size);
 cudaMemcpy(Ad, A, size, cudaMemcpyHostToDevice);

 float* Bd;
 size = wA * wB * sizeof(float);
 cudaMalloc((void**)&Bd, size);
 cudaMemcpy(Bd, B, size, cudaMemcpyHostToDevice);

  // Allocate C on the device
 float* Cd;
 size = hA * wB * sizeof(float);
 cudaMalloc((void**)&Cd, size);

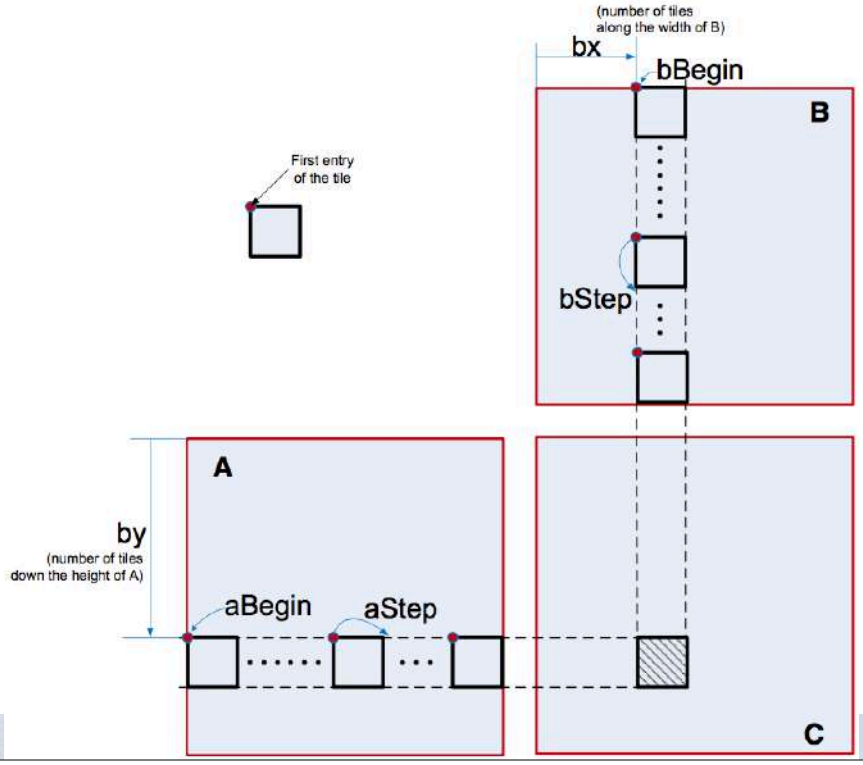
 // Compute the execution configuration assuming
 // the matrix dimensions are multiples of BLOCK_SIZE
 dim3 dimBlock (BLOCK_SIZE, BLOCK_SIZE);
 dim3 dimGrid (wB/dimBlock.x, hA/dimBlock.y);

  // Launch the device computation
 Muld<<<dimGrid, dimBlock>>>(Ad, Bd, wA, wB, Cd);

  // Read C from the device
 cudaMemcpy(C, Cd, size, cudaMemcpyDeviceToHost);

  // Free device memory
 cudaFree(Ad);
 cudaFree(Bd);
 cudaFree(Cd);
}
```



Matrix Multiplication - Device Code

```
// Device multiplication function called by Mul()
// Compute C = A * B
// wA is the width of A
// wB is the width of B
__global__ void Muld(float* A, float* B, int wA, int wB, float* C){

  // Block index
 int bx = blockIdx.x; // the B & C matrix sub-block column index
 int by = blockIdx.y; // the A & C matrix sub-block row index

  // Thread index
 int tx = threadIdx.x; // the column index in the sub-block
 int ty = threadIdy.y; // the row index in the sub-block

  // Index of the first sub-matrix of A processed by the block
 int aBegin = wA * BLOCK_SIZE * by;
  // Index of the last sub-matrix of A processed by the block
 int aEnd = aBegin + wA - 1;
  // Step size used to iterate through the sub-matrices of A
 int aStep = BLOCK_SIZE;
  // Index of the first sub-matrix of B processed by the block
 int bBegin = BLOCK_SIZE * bx;
  // Step size used to iterate through the sub-matrices of B
 int bStep = BLOCK_SIZE * wB;
  // The element of the block sub-matrix that computed by the thread
 float Csub = 0;
  // Shared memory for the sub-matrix of A
 __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
  // Shared memory for the sub-matrix of B
 __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];
 // Loop over all the sub-matrices of A and B required to compute
 // the block sub-matrix
 for (int a = aBegin, b = bBegin;
  a <= aEnd;
  a += aStep, b += bStep){
     // Load the matrices from global memory to shared memory;
     // each thread loads one element of each matrix
  As[ty][tx] = A[a + wA * ty + tx];
  Bs[ty][tx] = B[b + wB * ty + tx];
     // Synchronize to make sure the matrices are loaded
  __syncthreads();
     // Multiply the two matrices together; each thread computes
     // one element of the block sub-matrix
  for (int k = 0; k < BLOCK_SIZE; ++k)
    Csub += As[ty][k] * Bs[k][tx];
     // Synch to make sure the preceding computation is done before
     // loading two new sub-matrices of A and B in the next iteration
  __syncthreads();
  }
     // Write the block sub-matrix to global memory;
     // each thread writes one element
  int c = wB * BLOCK_SIZE * by + BLOCK_SIZE * bx;
  C[c + wB * ty + tx] = Csub;
}
```

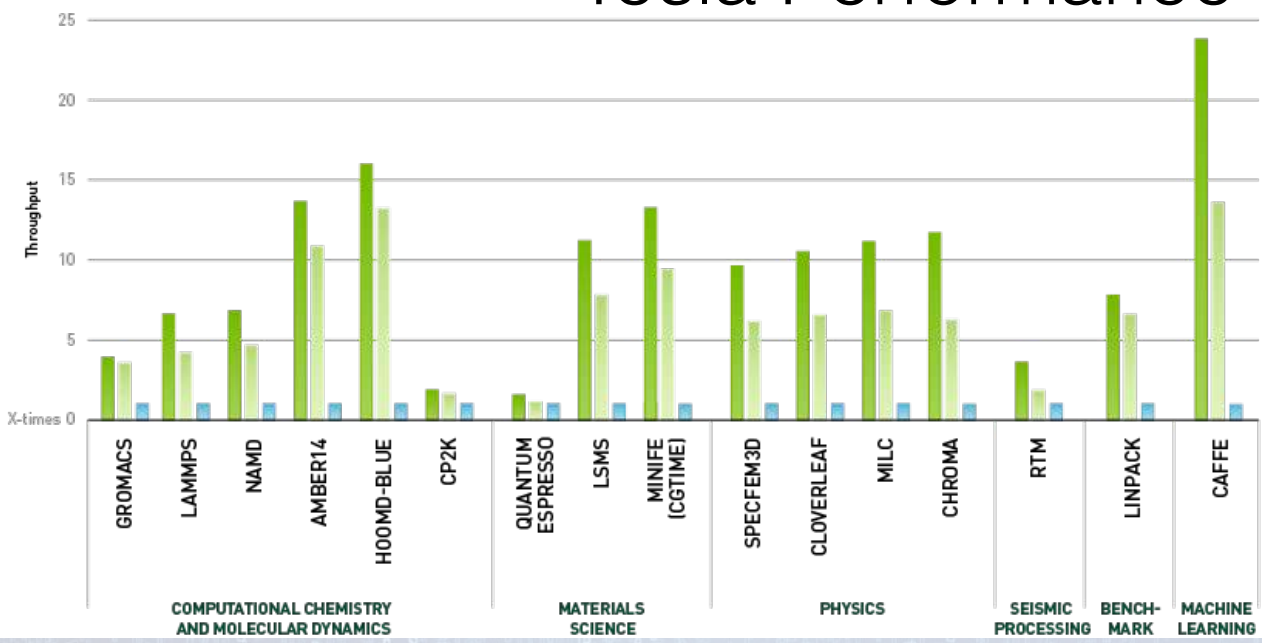
Further Matrix-Multiply Performance

- Different implementations
  - Reference **kij**-ordered version on a 3GHz Xeon SC is 1.7 GFLOPS
  - V1 SIMD 32 i loop: 27 GFLOPS
  - V2 SIMD 32 k loop + Local Memory: 44 GFLOPS
  - V3 unroll 2 values from i loop: 43 GFLOPS
  - V4 unroll 2 values from j loop: 169 GFLOPS
  - V3+V4: 173 GFLOPS
- The bottom line
  - **Relatively easy to get something to run on GPUs**
  - **Difficult to get it to run fast…** (e.g. algorithms, memory, I/O, etc.)

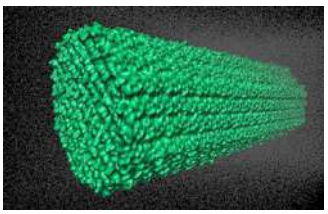


Performance

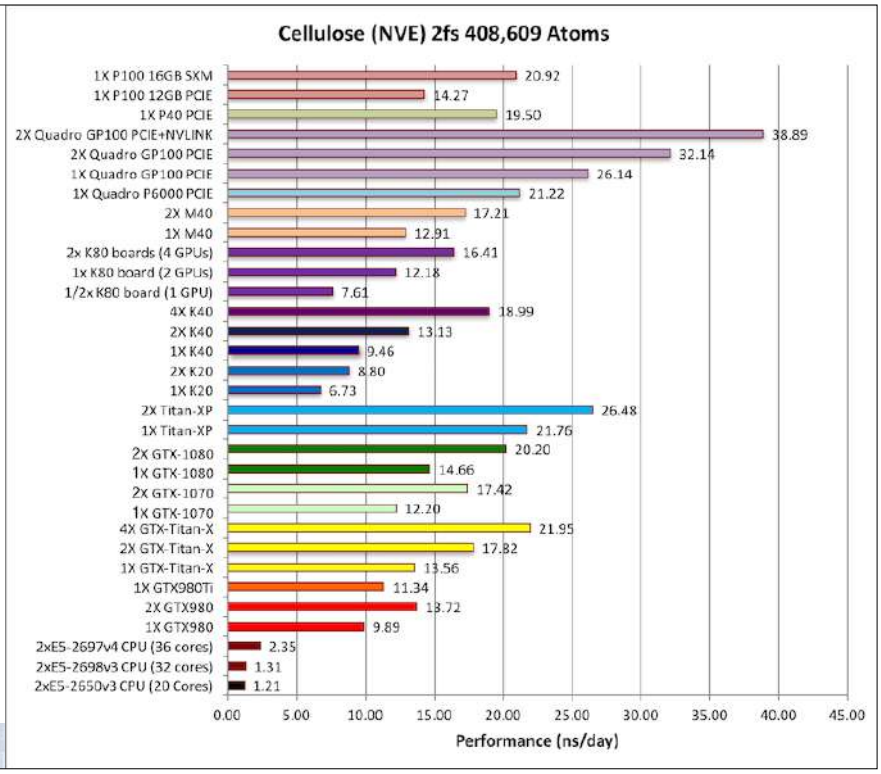
Tesla Performance



Amber Molecular Simulations



Cellulose

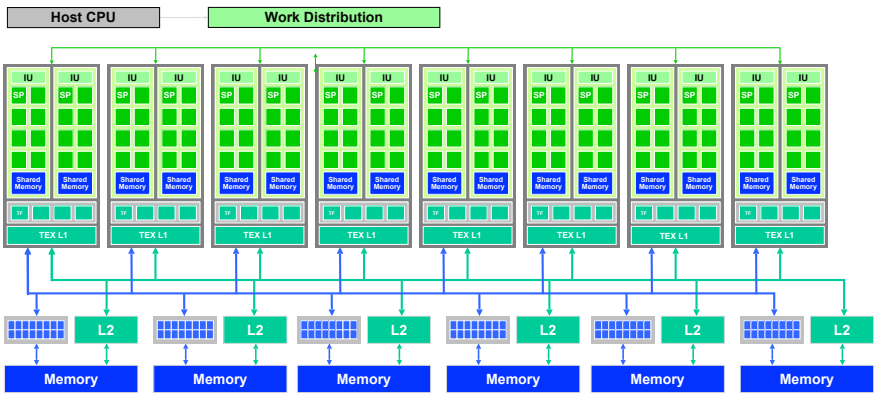




Going Forward

The Future of Computing is Parallel

- CPU clock rate growth is ~slowing, future computing growth will be from parallelism
- GPU Accelerators are a massively parallel computing platforms
  - 12,288 concurrent threads, hardware managed
  - 128 SP Thread Processor cores at 1.35 GHz == 518 GFLOPS peak



Implications and Opportunities

- Massively parallel computing allows
  - Drastic reduction in “time to discovery”
  - The new 3rd paradigm for research: computational experimentation
  - The “democratization of supercomputing”
    - $1,000/Teraflop in personal computers today
    - $1,000,000/Petaflops in clusters today
    - HW cost will no longer be the main barrier for big science
    - Global competition will be won with abilities to create and use parallel systems for discovery
- Future winner academic institutions will be leaders in
  - Research in Parallel Programming and Parallel Architecture
  - Teaching massively parallel programming to CS/EE students & scientists



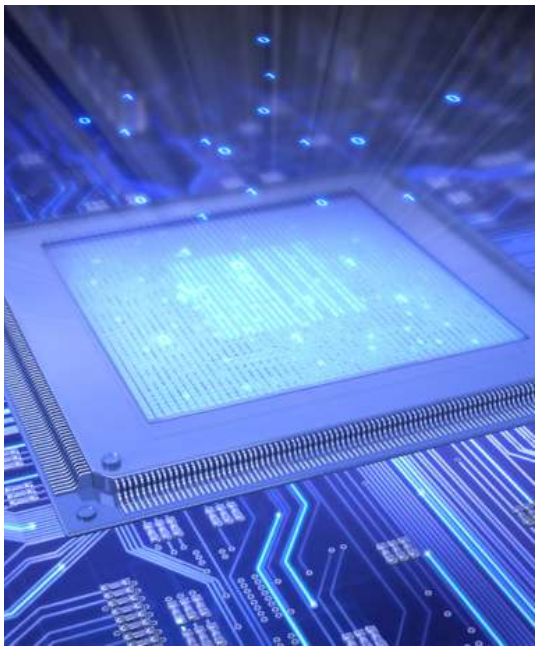
Ce am invatat in aces curs?

- The Roofline Model
- GPU Architecture
- GPU Programming
- GPU Performance
- Going Forward



Slides & Support Material Acknowledgements

Elsevier Inc. 2012
Ye Mesman Corporaal - 2010
Sathish Vadhiyar - 2009
David Patterson - Computer Architecture is Back 2007
Michael Garland. David B. Kirk, "Understanding throughput-oriented architectures", CACM 2010
David Kirk - NVidia 2014-2021
Dan Negrut & Dan Melanz & Andrew Seidl - 2013
Michael Wolfe - Portland Compiler Group - 2009
Robert Strzodka - Max Plank Institute - 2011
Cache-Aware Roofline Model - 2018



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro